



MANUAL

MIPSIM USER MANUAL

ABSTRACT

The MIPSIM manual, used to quickly understand how it works, explain instructions and individual functionalities.

Martin Gregor

Supervisor: Ing. Ján Hudec, PhD.

Content

Program	2
Phase	2
Conflict resolution	2
Basic functionalities.....	3
Notation.....	3
Observation	3
Control Panel	4
Time part	4
File part.....	4
Information part	4
Instructions.....	5
Arithmetic instructions.....	5
Bit Logical instructions.....	5
Jumping instructions	6
Memory instructions	6
Shifting instructions.....	7
Special instructions.....	7

Program

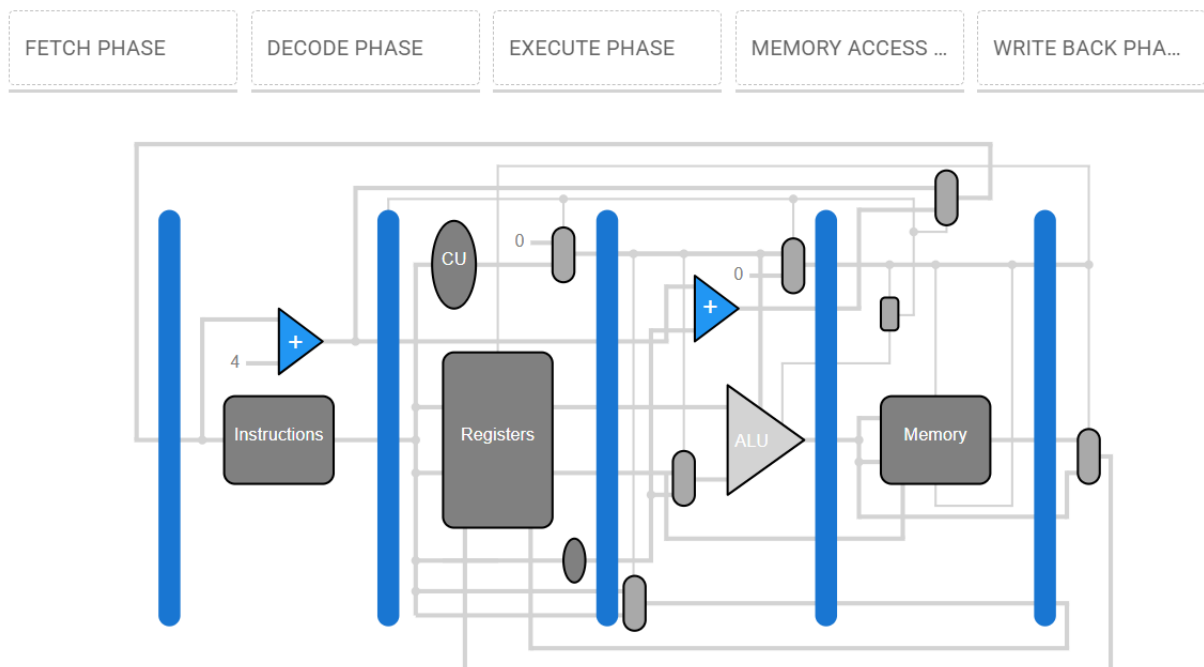
The program is built using the **Quasar** and **Vue** frameworks. The Pinia library was used to manage states and link components. Clipchamp was used to edit the video **included in the program**, along with the **Play.ht** platform, which provided voice synthesis and subtitles.

Phase

The work documentation describes in detail each step of the programme's development, as well as its overall functioning. In a nutshell, the program is divided into five phases: **Fetch**, **Decode**, **Execute**, **Memory Access**, and **Write Back**.

- **Fetch** – the program loads individual instructions so that it knows what to do in the next phases.
- **Decode** – the instruction is decoded and the necessary registers or memory data are retrieved if necessary.
- **Execute** – arithmetic or logical operations are performed using the ALU.
- **Memory Access** – writes data back to memory if the instruction requires it.
- **Write Back** – the results are written back to the registers.

These phases run in parallel, allowing for **current instruction processing (pipelining)**, increasing the efficiency of program execution.



Conflict resolution

A conflict can occur in the normal operation of a program – while one instruction reads a value from memory or register, another tries to write this value at the same time. This problem is solved in some programs by inserting the so-called **NOP instructions** that serve as a "pause" to eliminate the collision.

However, this problem does not arise in our program. Although the individual phases of instruction processing take place simultaneously (as part of stream processing), they do not have the same

priority. **Priorities are set from the end of the pipeline**, which means that the write phase takes precedence over read.

In other words: if a conflict occurs, **the value is first written** to the register or memory and only then is it allowed to be read by another instruction. In this way, data consistency is ensured without the need for additional interventions in the management of implementation.

This does not mean that it is not necessary to use NOPs at all across phases, but thanks to priorities, we can reduce their number.

Basic functionalities

The program can be divided into three main parts:

1. **Writing part** - contains registers, memory and a list of instructions. The user interacts with individual fields, which he is able to rewrite during the program itself.
2. **Observation part** - includes a diagram of MIPSIM and visualization of individual phases, including a display of which phase a particular instruction is currently in.
3. **Control Panel** – is further divided into three subsections:
 - a. Time part
 - b. File part
 - c. Information part

Notation

The user can write to registers, memory, and instruction lists at any stage of the simulation. Unlike previous versions of MIPSIM, if the user enters the wrong syntax, uses a non-existent register, label or memory address, the program does not crash. **The simulation continues** with an **error message** and the erroneous instruction is **processed as NOP** (no operation) in the program's internal logic.

This ensures that the user **does not have to manually overwrite every invalid or temporarily deleted instruction on the NOP** – it is enough to leave the field blank or with a misspelling. The same applies to registers and memory – if the user deletes a value (e.g. leaves the field blank instead of zero), the program will not crash and the simulation will continue.

It is important to emphasize that in MIPSIM, **memory addresses are divided by 4 bytes**, i.e. the first address is 0000, the second is 0004, etc. However, to simplify the work, the user is shown **the real value of the address** in the upper left corner of the memory table. **The user works with the system in the decimal system**, which is more natural and clearer. Nevertheless, it still has the opportunity to see **how the data is actually stored** in memory according to the real addressing of the MIPS architecture.

Observation

In this section, the user is in a passive role and has limited opportunities to interact with the environment itself. It serves mainly as a visual aid to illustrate how the program works – what units communicate with each other and how they interact with memory, registers, instruction set, arithmetic-logic unit...

Control Panel

Probably the most important element of the whole program is the so-called **control panel**. It is used to manipulate the entire code and then transform it into the desired form. As mentioned earlier, **the dashboard is divided into three sections**.



Time part

The middle part of the control panel is used to control the simulation. In it, the user can choose from **three different speeds of program execution**. If they don't want the program to run automatically, they have the option **to manually step through** it – that is, scroll instruction by instruction.

The program can be **stopped** or **reset** at any point in the simulation:

- **Stop** – suspends the program, but **maintains the current state of all components** (instructions, memory, registers).
- **Reset** – **preserves the contents of memory, registers and instructions**, but resets **Program Counter**. After restarting the program, it will start executing instructions from the beginning.

At the same time, the user has **the option to independently delete the contents of registers or memory** – for example, when testing a program, when he needs to set the registers back to zero or empty the memory field to which the program should write.

File part

The right part of the control panel is used to work with files. The user has the option **to import or export** the current state of instructions, memory, and registers. The program works with files in **JSON format**.

We recommend that users **do not edit these files outside of the MIPSIM environment** unless they are experienced enough. In case of incorrect handling (e.g. changing the number of registers or memory size), **the program may not be able to load the file correctly**.

This section also includes **a button to create a completely new program** that **resets the registers, memory, instructions, and Program Counter** – essentially starting a clean working environment.

Information part

In the last, left part of the interface, there is a list of all instructions. Here, the user can **view a detailed description, explanation, and syntax of each individual instruction**.

In addition, there is also **a settings section** where the user can choose **in which number system the values** in the registers and memory will be displayed – for example, **decimal, hexadecimal or binary**. There is also the **option to change the interface language**, which is ready for future implementation (due to the size of the library), but is not yet fully functional in the current version.

The last part of this section is **the Help**, which includes:

- The **program manual itself**,

- **video tutorial** as a light-hearted form of teaching, supplemented by subtitles and commentary,
- and a **brief description of the program** to help new users quickly find their way around.

Instructions

The instructions are also described in detail directly in the program. However, for a better overview and coherence, they are also listed in this document – for those who prefer to read instructions in PDF format rather than directly in the application.

Instructions can be divided into arithmetic, bit, jump, memory, shifting and special.

Arithmetic instructions

Instruction	Syntax	Description
ADD (Addition)	ADD \$Rx \$Ry \$Rz	The instruction adds the registers Ry and Rz. Stores the result in the register Rx.
ADDI (Addition Immediate)	ADDI \$Rx \$Ry \$i	The instruction adds the register Ry and the constant i. Stores the result to the register Rx.
SUB (Subtraction)	SUB \$Rx \$Ry \$Rz	The instruction subtracts the registers Ry and Rz. Stores the result in the register Rx.
SUBI (Subtraction Immediate)	SUBI \$Rx \$Ry \$i	The instruction subtracts the register Ry and the constant i. Stores the result to the register Rx.
MUL (Multiply)	MUL \$Rx \$Ry \$Rz	The instruction multiplies the registers Ry and Rz. Stores the result in the register Rx.
MULI (Multiply Immediate)	MULI \$Rx \$Ry \$i	The instruction multiplies the register of Ry and the constant i. Stores the result to the Rx register.
MULU (Multiply Unsigned)	MULU \$Rx \$Ry \$Rz	The instruction multiplies the registers Ry and Rz. Stores the result in the register Rx. (Unsigned)
DIV (Divide)	DIV \$Rx \$Ry \$Rz	The instruction divides the register Ry by the register Rz. Stores the result in the register Rx.
DIVI (Divide Immediate)	DIVI \$Rx \$Ry \$i	The instruction divides the register Ry by the constant i. Stores the result in the register Rx.
DIVU (Divide Unsigned)	DIVU \$Rx \$Ry \$Rz	The instruction divides the register Ry by the register Rz. Stores the result in the register Rx. (Unsigned)

Bit Logical instructions

Instruction	Syntax	Description
AND (Bitwise AND)	AND \$Rx \$Ry \$Rz	Bitwise AND between the registers Ry and Rz. Stores the result in the register Rx.
ANDI (Bitwise AND Immediate)	ANDI \$Rx \$Ry \$i	Bitwise AND between register Ry and constant i. Stores the result to the Rx register.
OR (Bitwise OR)	OR \$Rx \$Ry \$Rz	Bitwise OR between the registers Ry and Rz. Stores the result in the register Rx.
ORI (Bitwise OR Immediate)	ORI \$Rx \$Ry \$i	Bitwise OR between register Ry and constant i. Stores the result to the Rx register.

XOR (Bitwise XOR)	XOR \$Rx \$Ry \$Rz	Bitwise XOR between the registers Ry and Rz. Stores the result in the register Rx.
XORI (Bitwise XOR Immediate)	XORI \$Rx \$Ry \$i	Bitwise XOR between register Ry and constant i. Stores the result to the Rx register.
NAND (Bitwise NAND)	NAND \$Rx \$Ry \$Rz	Bitwise negated AND between the registers Ry and Rz. Stores the result in the register Rx.
NANDI (Bitwise NAND Immediate)	NANDI \$Rx \$Ry \$i	Bitwise negated AND between register Ry and constant i. Stores the result in register Rx.
NOR (Bitwise NOR)	NOR \$Rx \$Ry \$Rz	Bitwise negated OR between the registers Ry and Rz. Stores the result in the register Rx.
NORI (Bitwise NOR Immediate)	NORI \$Rx \$Ry \$i	Bitwise negated OR between register Ry and constant i. Stores the result in register Rx.
XNOR (Bitwise XNOR)	XNOR \$Rx \$Ry \$Rz	Bitwise negated XOR between the registers Ry and Rz. Stores the result in the register Rx.
XNORI (Bitwise XNOR Immediate)	XNORI \$Rx \$Ry \$i	Bitwise negated XOR between register Ry and constant i. Stores the result in register Rx.

Jumping instructions

Inštrukcia	Syntax	Popis
BEQ (Branch if Equal)	BEQ \$Rx \$Ry \$instruction	The instruction will jump to the label if the Rx register is equal to the Ry register.
BNEQ (Branch if Not Equal)	BNEQ \$Rx \$Ry \$instruction	The instruction will jump to the label if the Rx register is not equal to the Ry register.
BHI (Branch if Higher)	BHI \$Rx \$Ry \$instruction	The instruction will jump to the label if the Rx register is higher than the Ry register.
BLO (Branch if Lower)	BLO \$Rx \$Ry \$instruction	The instruction will jump to the label if the Rx register is lower than the Ry register.
J (JUMP)	J \$instruction	The instruction performs an unconditional jump to the signal.

Memory instructions

Inštrukcia	Syntax	Popis
LW (Load Word)	LW \$Rx \$Ry \$memory	The instruction loads a word from memory at the address specified by the sum of the value in the register Ry and the offset. Stores the read word in the Rx register.
LWI (Load Word Immediate)	LWI \$Rx \$i \$memory	The instruction loads a word from memory at the address specified by the sum of the value of the constant i and the offset. Stores the read word in the Rx register.
SW (Store Word)	SW \$Rx \$Ry \$memory	The instruction stores the word in memory at the address specified by the sum of the value in the register Ry and the offset. The stored word is from the Rx register.
SWI (Store Word Immediate)	SWI \$Rx \$i \$memory	The instruction stores the word in memory at the address specified by the sum of the value of the

		constant i and the offset. The stored word is from the Rx register.
--	--	---

Shifting instructions

Inštrukcia	Syntax	Popis
SLLV (Shift Left Logical Variable)	SLLV \$Rx \$Ry \$Rz	The instruction shifts the bit value of a number in the register Ry by the number of positions specified by the value in the register Rz to the left. Stores the result in the Rx register.
SRLV (Shift Right Logical Variable)	SRLV \$Rx \$Ry \$Rz	The instruction shifts the bit value of a number in the register Ry by the number of positions specified by the value in the register Rz to the right. Stores the result in the Rx register.
SLLVI (Shift Left Logical Variable Immediate)	SLLVI \$Rx \$Ry \$i	The instruction shifts the bit value of a number in the register Ry by the number of positions specified by the value of the constant i to the left. Stores the result to the Rx register.
SRLVI (Shift Right Logical Variable Immediate)	SRLVI \$Rx \$Ry \$i	The instruction shifts the bit value of a number in the register Ry by the number of positions specified by the value of the constant i to the right. Stores the result to the Rx register.

Special instructions

Inštrukcia	Syntax	Popis
NOP (No Operation)	NOP	An instruction that does not perform any operation.
LI (Load Immediate)	LI \$Rx \$i	The instruction reads the constant i and stores it in the register Rx.
LUI (Load Upper Immediate)	LUI \$Rx \$i	The instruction reads the top 16 bits of the constant i and stores it in the Rx register.
Q (QUIT)	Q	Early termination of the program.